GENERATIVE MODELS

Complete Study Notes | Exam Ready

Deep Learning Unit — Concepts · Formulas · Examples · Exam Focus

UNIT OVERVIEW

UNIT 1: FUNDAMENTALS OF GENERATIVE MODELS

▸ Generative vs Discriminative Models

Feature	Details
Generative Models	Learn the FULL joint distribution P(X, Y) — can generate new data samples
Discriminative Models	Learn the conditional P(Y|X) — only classify/predict labels
Goal of Generative	Model how data is generated; sample new realistic data
Examples — Generative	VAE, GAN, Normalizing Flows, Diffusion Models
Examples — Discriminative	Logistic Regression, SVM, CNN classifiers

💡 **Key Intuition:**

DISC	Given an image → Is it a cat or dog? (classification only)

GEN	Learn what cats & dogs look like → Generate new cat images!

▸ Probability Basics

Joint Distribution P(X, Y)

- Describes the probability of X and Y occurring together
- Example: P(image of cat, label='cat') — how likely to see this image with this label
- Generative models learn this full distribution

Likelihood P(X|θ)

- Probability of observing data X given model parameters θ
- Training goal: find θ that maximizes P(X|θ) — called Maximum Likelihood Estimation (MLE)

$$\theta^* = \text{argmax}_\theta \sum \log P(x_i \mid \theta)$$

▸ Density Estimation

Type	Explicit Models	Implicit Models
Definition	Directly define & compute P(X)	Learn to sample from P(X) without specifying it
Examples	VAE, Normalizing Flows	GAN
Advantage	Can evaluate likelihood	More flexible, high-quality samples
Disadvantage	Often intractable	Cannot evaluate exact likelihood

Latent Variable Models

- Data X is explained by hidden (latent) variables Z
- Model: $P(X) = \int P(X|Z)\, P(Z)\, dZ$

Latent Space

Definition	A compressed hidden representation of data in a lower-dimensional space
Continuity	Nearby points in latent space → similar outputs (smooth space)
Interpolation	Can blend between two points: $z = 0.5 \cdot z_1 + 0.5 \cdot z_2$ → generates in-between image

Encoder & Decoder Mapping

INPUT DATA (e.g. image 28×28 pixels)

▼

ENCODER → data → latent vector z

▼

LATENT SPACE z (compressed representation)

▼

DECODER → latent z → reconstructed data

▼

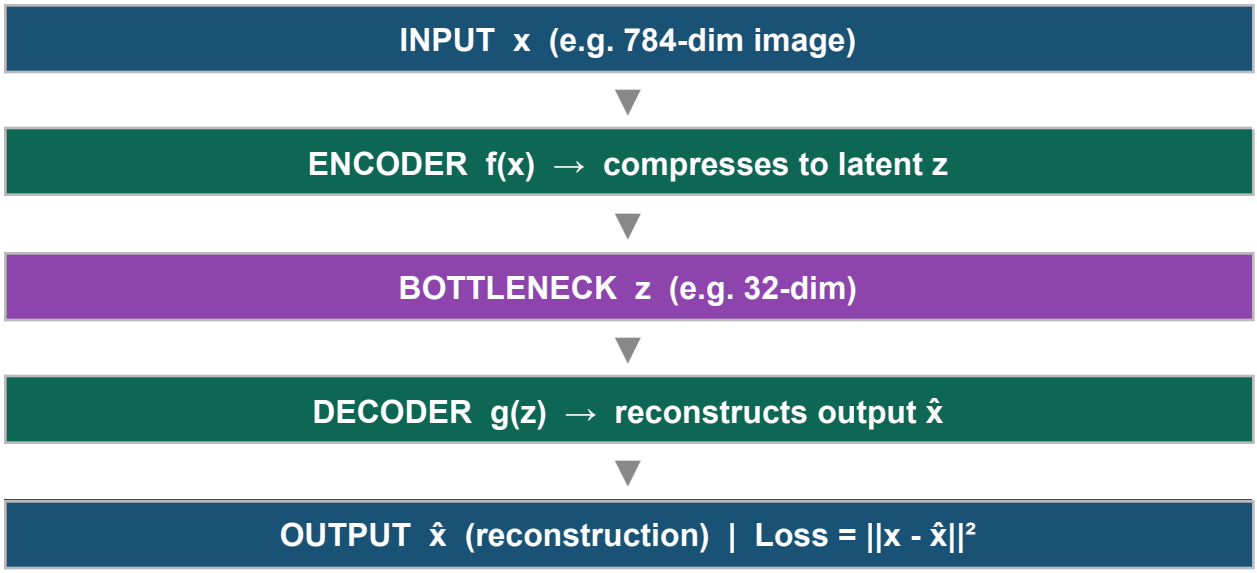
OUTPUT (reconstructed / generated image)

📋 Exam Focus — Unit 1

★	Likelihood maximization: know the MLE formula and what 'maximizing log-likelihood' means
★	Distinguish joint P(X,Y) vs conditional P(Y|X) vs likelihood P(X|θ) with examples
★	Explicit vs Implicit density estimation — identify which model type belongs to which
★	Latent space: what is it? Properties of continuity and interpolation
★	Encoder maps data → latent space; Decoder maps latent → data

UNIT 2: AUTOENCODERS (AE)

Architecture

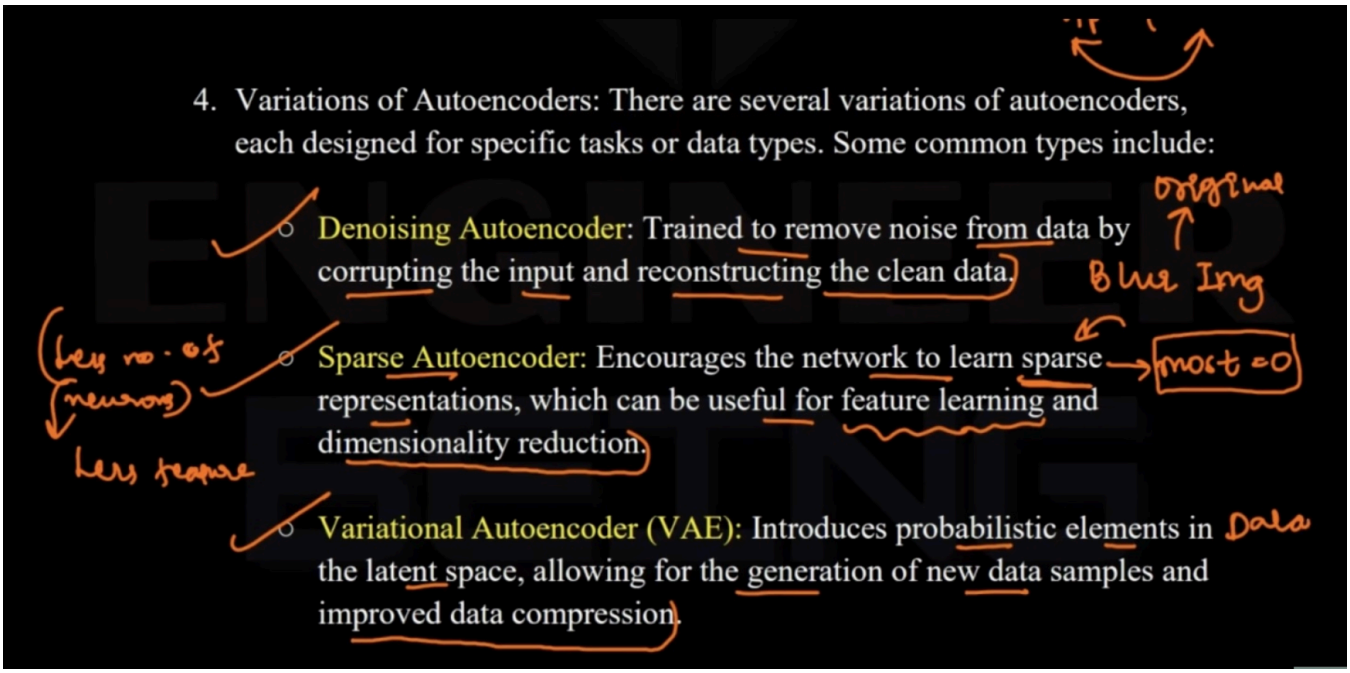


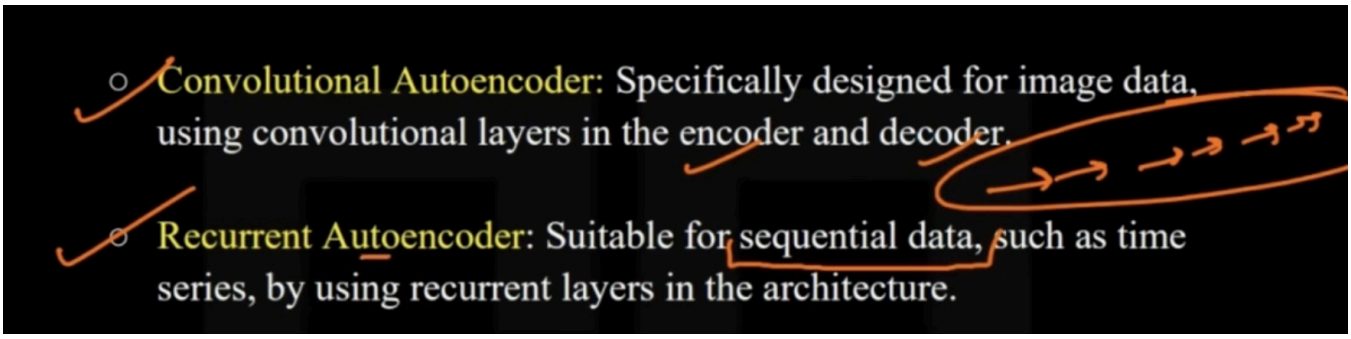
Key concept: The bottleneck forces the network to learn a compact, meaningful representation of the data.

EXAMPLE	Input: 28×28 MNIST digit (784 pixels) → Encoder → 32-dim bottleneck → Decoder → 784-pixel reconstruction

Types of Autoencoders

Type	Key Idea	Use Case
Vanilla AE	Basic encoder-decoder; minimize reconstruction loss	Dimensionality reduction, basic compression
Sparse AE	Forces most latent neurons to be 0 (sparsity penalty added)	Feature extraction, learns interpretable features
Denoising AE	Input: corrupted/noisy data; Target output: clean original data	Noise removal, robust feature learning
Contractive AE	Adds Frobenius norm of Jacobian to loss — small input changes → small latent changes	Robust representations, manifold learning





Loss Functions

$$\text{MSE Loss} = \frac{1}{n}\sum (x_i - \hat{x}_i)^2 \quad \leftarrow \text{use for continuous data (real-valued)}$$

$$\text{BCE Loss} = -\sum [x \cdot \log(\hat{x}) + (1-x) \cdot \log(1-\hat{x})] \quad \leftarrow \text{use for binary data (0 or 1)}$$

Loss Type	When to Use
MSE (Mean Squared Error)	Output is continuous, e.g. pixel values in [0,1] range
BCE (Binary Cross Entropy)	Output is binary, e.g. black/white images, 0/1 pixels

Applications

- **Data compression**: reduce 784-dim → 32-dim, store/transmit efficiently
- **Noise removal**: train on noisy input → clean output (Denoising AE)
- **Feature extraction**: bottleneck features used for downstream classification

- **Anomaly detection**: high reconstruction error → data point is anomalous!

AE vs PCA Comparison

Property	Autoencoder	PCA
Type	Non-linear (neural network)	Linear projection
Flexibility	Learns complex features	Only captures linear variance
Reconstruction	Better for complex data	Optimal for linear data
Computation	Requires training (gradient descent)	Closed-form solution (SVD)
Use for	Images, audio, complex data	Simple datasets, preprocessing

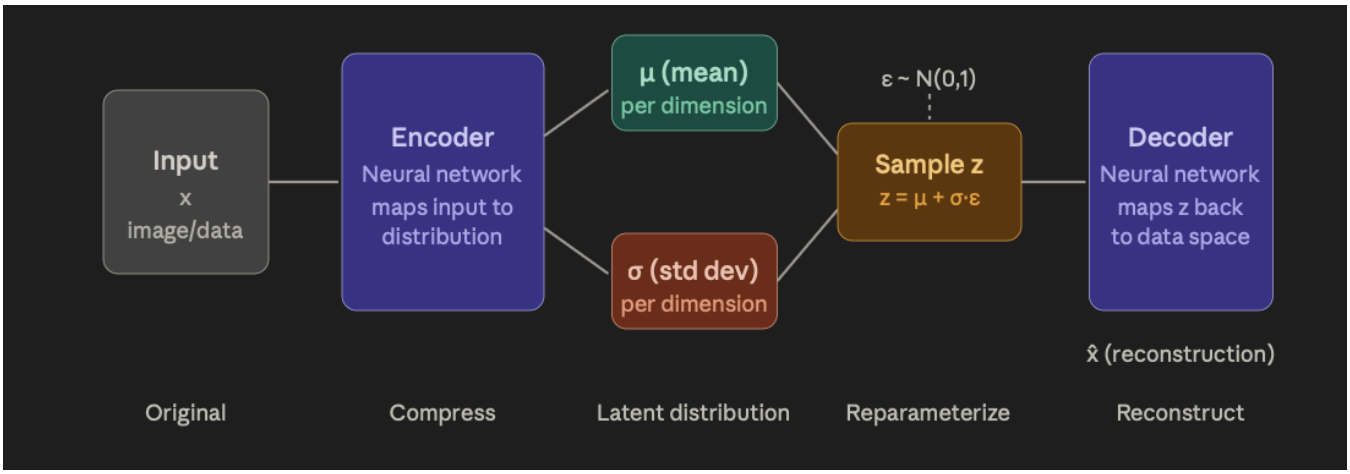
📋 Exam Focus — Unit 2

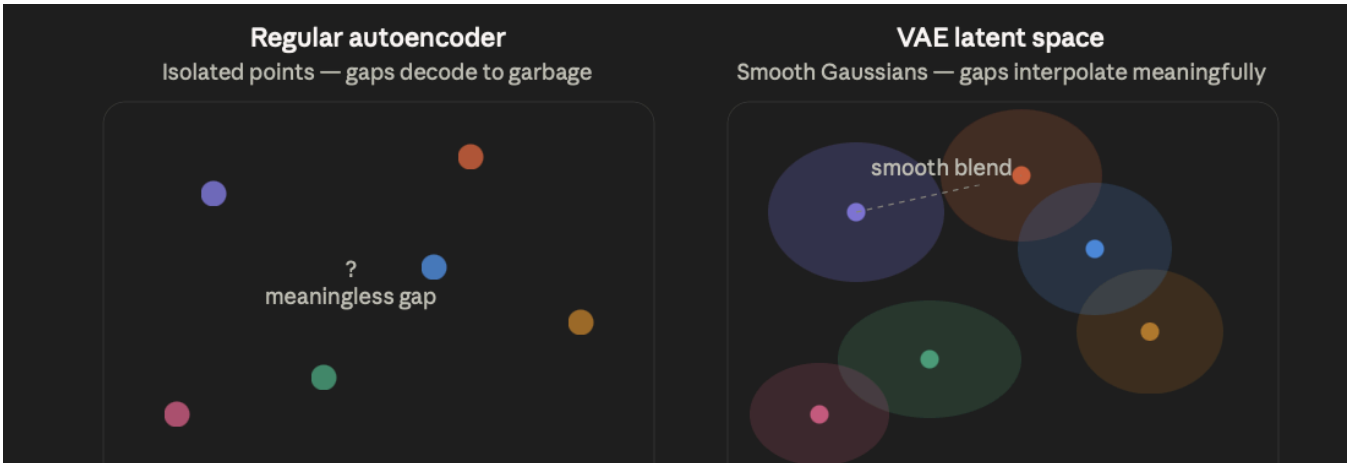
★	Reconstruction Loss: MSE = $(1/n)\Sigma(x - \hat{x})^2$ — be ready to compute for small examples
★	Compression Ratio = Input dimensions / Bottleneck dimensions (e.g. 784→32 = 24.5:1)
★	Parameter count: for layer $n_1 \rightarrow n_2$, params = $n_1 \times n_2 + n_2$ (weights + biases)
★	AE vs PCA: AE is non-linear (neural), PCA is linear (matrix); AE is generally better for images
★	Know all 4 types: Vanilla / Sparse / Denoising / Contractive — their key difference

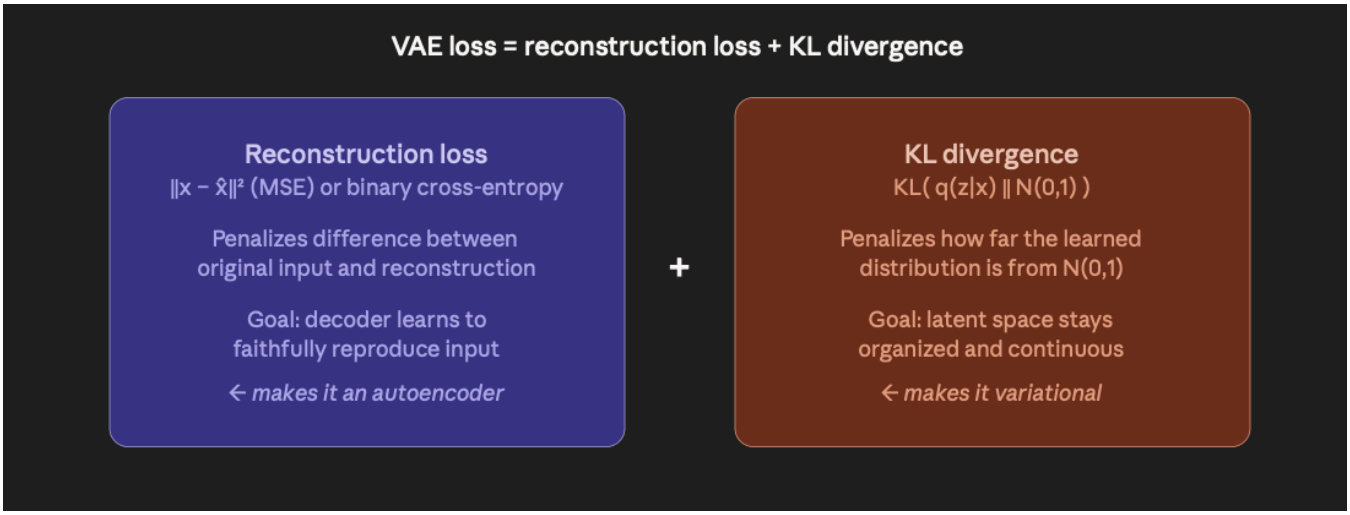
UNIT 3: VARIATIONAL AUTOENCODERS (VAE)

▸ Key Idea

VAE	Instead of encoding to a fixed point z, the encoder outputs a DISTRIBUTION over z (mean μ and variance σ²). We then SAMPLE from this distribution to generate z.







What problem does a VAE solve? A regular autoencoder compresses data to a point in latent space. The problem: those points are scattered with no structure between them — if you sample a random point, the decoder produces garbage. A VAE fixes this by encoding each input as a *distribution* (a Gaussian blob), not a point.

The encoding step — instead of outputting one vector z, the encoder outputs two vectors: μ (mean) and σ (standard deviation). These define a Gaussian distribution over latent space for each input.

The reparameterization trick — to sample $z = \mu + \sigma \cdot \varepsilon$ (where $\varepsilon \sim N(0,1)$), the trick separates the randomness into ε. This makes the operation differentiable, so gradients can flow back through the network during training — without this trick, you couldn't backpropagate through a sampling operation.

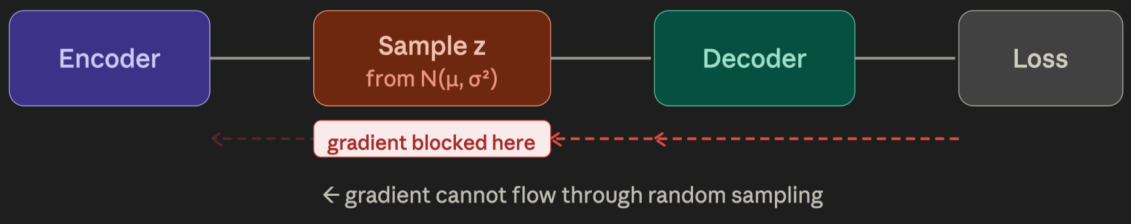
The KL divergence term acts like a regularizer. It pushes all the Gaussian blobs toward $N(0,1)$, so the latent space becomes dense and smooth. As a result, any point you sample from $N(0,1)$ decodes into something meaningful — this is what makes VAEs generative.

The tension between the two losses:

- Reconstruction loss says: *make your Gaussians narrow and precise* (encode each input distinctly)
- KL loss says: *make your Gaussians wide and centered at zero* (spread out and overlap)
- Training finds a balance — organized, overlapping blobs that the decoder can navigate smoothly

Generation — after training, throw away the encoder. Sample $z \sim N(0,1)$ and pass it through the decoder to generate new data that looks like your training distribution.

visualize: Reparameterization trick



During training, gradients need to flow backward from the loss all the way to the encoder (to update μ and σ). But there's a wall: the sampling step $z \sim N(\mu, \sigma^2)$ is a stochastic (random) operation. You can't differentiate through randomness — so gradients are blocked and the encoder can't learn.

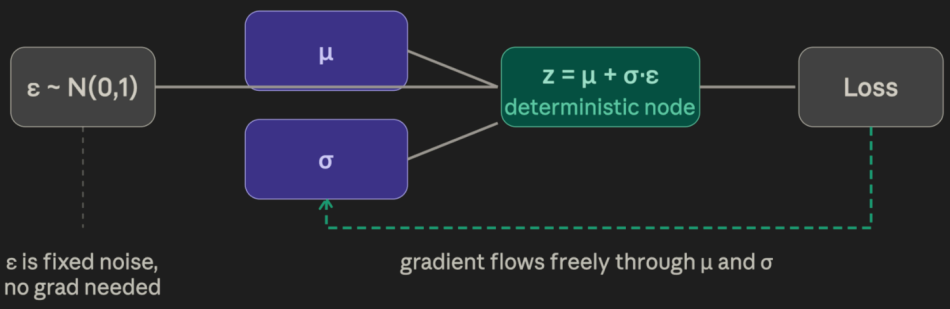
Instead of sampling z directly…

$z \sim N(\mu, \sigma^2)$
random, non-differentiable

…rewrite it as a deterministic formula:

$z = \mu + \sigma \cdot \varepsilon$, where $\varepsilon \sim N(0, 1)$
deterministic function of μ, σ, and fixed noise ε

The key rewrite: instead of sampling z from $N(\mu, \sigma^2)$ — which is random and non-differentiable — express z as a deterministic function: $z = \mu + \sigma \cdot \varepsilon$. The randomness is moved into a separate fixed noise variable ε sampled from a standard normal N(0,1). Now μ and σ are just parameters of a mathematical formula.



Now $z = \mu + \sigma \cdot \varepsilon$ is just arithmetic — a fully differentiable computation. The gradient of the loss flows backward through z, and since z depends on μ and σ via simple math, the chain rule works perfectly. The randomness lives in ε, which is treated as a fixed external input — no gradient needed there.

Both approaches produce the same distribution

Naive: $z \sim N(\mu, \sigma^2)$
correct samples, no gradients
encoder cannot be trained

Reparam: $z = \mu + \sigma \cdot \varepsilon$
same distribution of samples
encoder can be trained

The samples from $N(\mu, \sigma^2)$ and from $\mu + \sigma \cdot \varepsilon$ are statistically identical.
The trick is purely about making the computation graph differentiable.

The beautiful thing: $z = \mu + \sigma \cdot \varepsilon$ produces samples with exactly the same distribution as sampling from $N(\mu, \sigma^2)$. The math is equivalent. But now the encoder's parameters μ and σ are reachable by backpropagation, so the full VAE can be trained end-to-end with gradient descent.

AE	VAE
Encodes to a single point z	Encodes to a distribution $N(\mu, \sigma^2)$
Deterministic — same input → same z always	Probabilistic — same input → different z each time

AE	VAE
Latent space can have gaps (discontinuous)	Smooth, continuous latent space
Cannot easily generate new samples	Can sample z ~ N(0,1) to generate new data

▸ VAE Architecture & Components

INPUT x

▼

ENCODER → outputs μ(x) and σ²(x)

▼

SAMPLING z = μ + σ · ε where ε ~ N(0,1) [Reparameterization Trick]

▼

DECODER → reconstructs x̂ from z

▼

LOSS = Reconstruction Loss + KL Divergence

▸ Reparameterization Trick

❓ Problem: We cannot backpropagate through a random sampling step.

✅ Solution: Factor out the randomness!

📐 `z = μ + σ · ε  where ε ~ N(0, 1)  (standard normal noise)`

- **μ** and **σ** are outputs of the encoder (differentiable)
- **ε** is random noise sampled independently — not part of gradient graph
- This makes z **differentiable** w.r.t. encoder parameters → training works!

▸ VAE Loss Function (ELBO)

📐 `Total Loss = Reconstruction Loss + KL Divergence`

⭐ $KL(q(z|x) \,||\, p(z)) = -\frac{1}{2} \cdot \Sigma\,(1 + \log \sigma^2 - \mu^2 - \sigma^2)$

- **Formula:** The closed-form expression for KL divergence between a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ and the prior $\mathcal{N}(0,1)$ is:
  $KL(\mathcal{N}(\mu, \sigma^2)||\mathcal{N}(0,1)) = -0.5 \times \sum(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$

- **Asymmetry:** KL Divergence is not a true "distance" metric because it is asymmetrical: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. The order matters; P is typically the true distribution and Q is the approximation.
- **Non-negativity:** The value is always greater than or equal to zero, where $D_{KL}(P||Q) = 0$ if and only if P and Q are identical.
- **Formula (Discrete):** For discrete distributions P and Q:

$$D_{KL}(P||Q) = \sum_i P(i) \ln\left(\frac{P(i)}{Q(i)}\right)$$

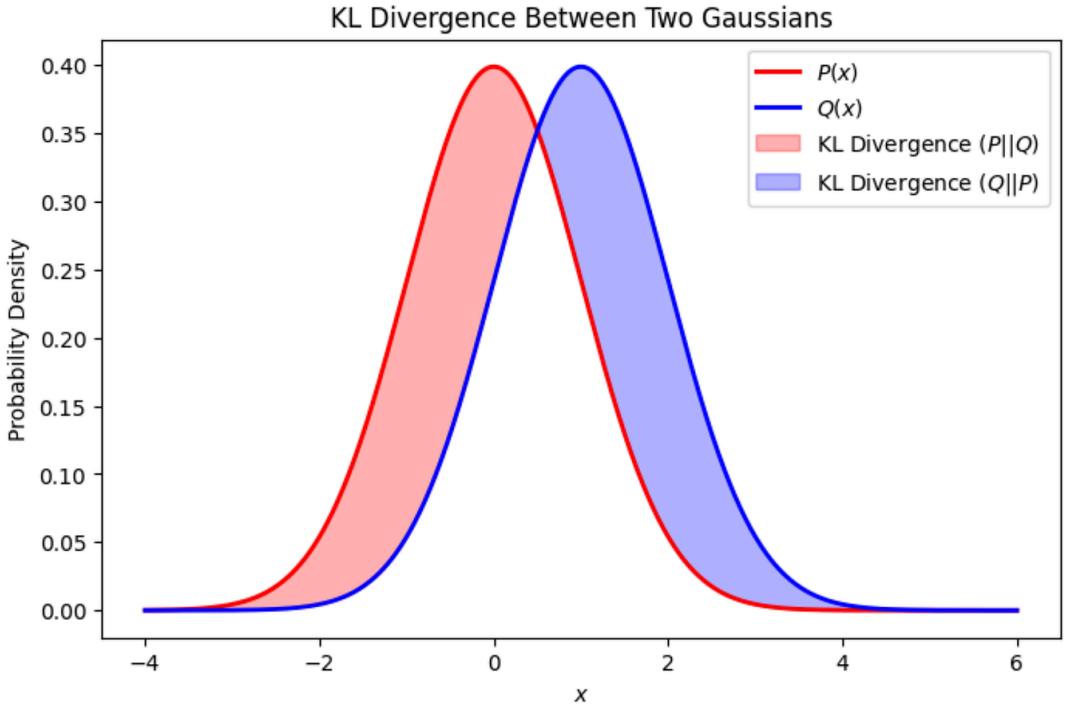
- **Formula (Continuous):** For continuous distributions P and Q:

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \ln\left(\frac{p(x)}{q(x)}\right) dx$$

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

$$D_{\mathrm{KL}}(P\|Q) = H(P,Q) - H(P)$$

$$\text{KL Divergence} = \text{Cross Entropy}(P,Q) - \text{Entropy}(P)$$



Easy Numerical Example

Suppose we have 2 possible outcomes:

Outcome	P(x) (True)	Q(x) (Approx)
A	0.8	0.6
B	0.2	0.4

Use formula:

$$DKL(P||Q) = \sum P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

Step 1: For Outcome A

$$0.8 \log\left(\frac{0.8}{0.6}\right)$$

$$= 0.8 \log(1.333)$$

Using natural log:

$$\log(1.333) \approx 0.287$$

$$0.8 \times 0.287 = 0.2296$$

Step 2: For Outcome B

$$0.2 \log\left(\frac{0.2}{0.4}\right)$$

$$= 0.2 \log(0.5)$$

$$\log(0.5) \approx -0.693$$

$$0.2 \times (-0.693) = -0.1386$$

Step 3: Add Both

$$DKL = 0.2296 + (-0.1386)$$

$$= 0.091$$

Final Answer:

$$DKL(P||Q) \approx 0.091$$

Reconstruction Loss	How well does the decoder reconstruct the original input? (MSE or BCE)
KL Divergence	Regularises latent space — keeps q(z|x) close to standard normal N(0,1). Prevents overfitting & ensures smooth space.

📌 ELBO = Evidence Lower BOund = −Total VAE Loss (maximize ELBO = minimize loss)

📋 Exam Focus — Unit 3

★	Reparameterization trick formula: z = μ + σ·ε where ε ~ N(0,1) — why it's needed (gradient flow)
★	KL Divergence formula and meaning: measures how far learned distribution is from N(0,1)
★	Total VAE loss = Reconstruction + KL — know what each term does
★	ELBO concept: Evidence Lower Bound — maximising ELBO trains the VAE
★	Difference between AE (point encoding) and VAE (distribution encoding)

UNIT 4: GENERATIVE ADVERSARIAL NETWORKS (GANs)

GAN Structure

GENERATOR G	DISCRIMINATOR D
• Takes random noise z ~ N(0,1) • Produces fake data G(z) • Goal: fool the Discriminator • Like: the Counterfeiter 🎨	• Takes real or fake images • Outputs probability P(real) • Goal: distinguish real vs fake • Like: the Detective 🔍

GAN Objective Function (Minimax)

$$\min_G \max_D V(D,G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

$\mathbb{E}[\log D(x)]$	Discriminator's expected log probability of correctly identifying REAL data as real. D wants this HIGH.
$\mathbb{E}[\log(1-D(G(z)))]$	Probability of correctly identifying FAKE data. D wants this HIGH; G wants D(G(z)) close to 1 so this term is LOW.

Training Steps (Adversarial)

Step 1: Sample real data x from dataset

▼

Step 2: Sample noise z ~ N(0,1); Generate fake data G(z)

▼

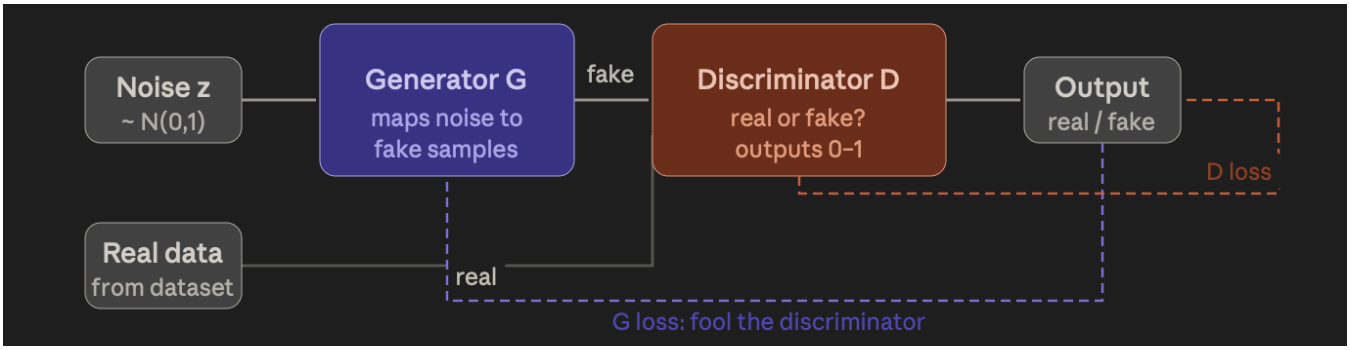
Step 3: Train DISCRIMINATOR — maximize log D(x) + log(1−D(G(z)))

▼

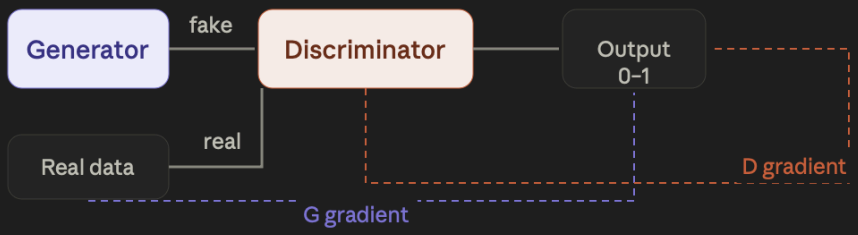
Step 4: Train GENERATOR — minimize log(1−D(G(z))) i.e. fool D

▼

Repeat until Nash Equilibrium: D(x) = 0.5 for all inputs



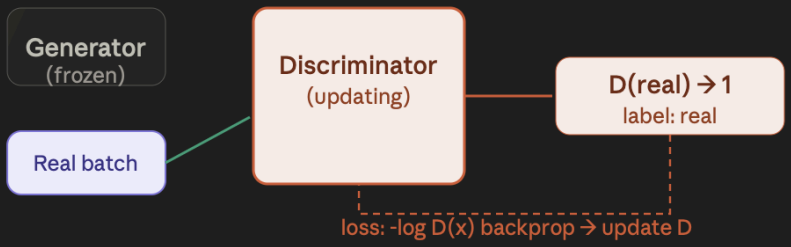
Training loop



GAN training alternates between two phases each iteration: first train the Discriminator (D) to get better at spotting fakes, then train the Generator (G) to get better at fooling D. Both networks update separately — their weights are frozen when it's not their turn.

```
Each iteration: [Step 1] Update D → [Step 2] Update G → repeat
```

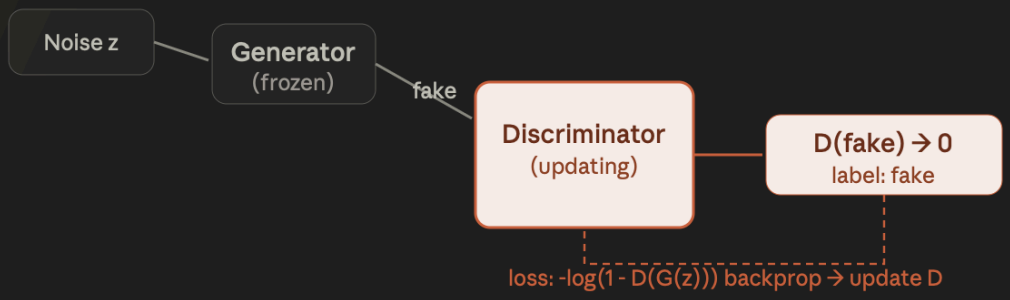
Discriminator phase — real samples



Feed a batch of real images from the training set into D. D should output a value close to 1 (real). The loss penalizes D for outputting low values on real data. Gradients flow back and D's weights update. G is frozen — it doesn't participate yet.

```
Loss_D_real = −log( D(x) ) ← want D(x) → 1
```

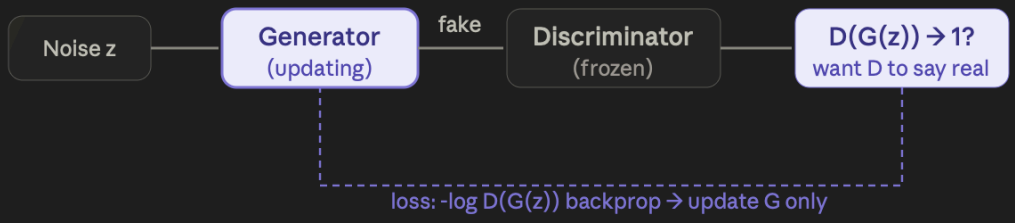
Discriminator phase — fake samples



Now feed a batch of fakes from G into D. D should output a value close to 0 (fake). The loss penalizes D for outputting high values on fakes. Both real and fake losses are combined and D updates. G is still frozen — it didn't contribute to this update.

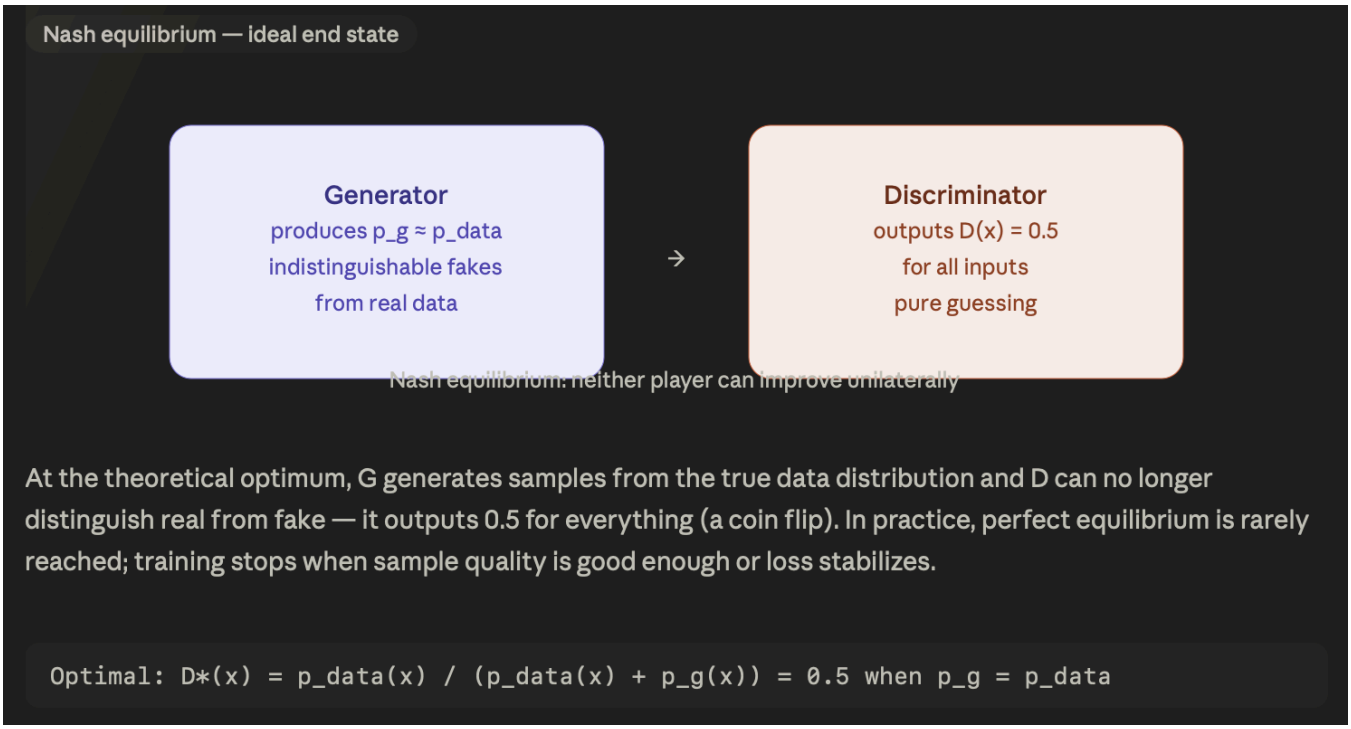
```
Loss_D_fake = −log( 1 − D(G(z)) ) ← want D(G(z)) → 0
```

Generator phase



Now freeze D and train G. Feed noise through G to get fakes, pass them through the frozen D, and compute a loss that penalizes G whenever D correctly identifies the output as fake. G's goal is to fool D — to get D(G(z)) close to 1. Gradients flow from D back through G's weights.

```
Loss_G = −log( D(G(z)) ) ← want D(G(z)) → 1
```



📋 Exam Focus — Unit 4

★	Know the minimax objective: min_G max_D V(D,G) = E[logD(x)] + E[log(1−D(G(z)))]
★	D output is a probability: D(x) ≈ 1 means 'real', D(G(z)) ≈ 0 means 'fake'
★	At equilibrium: D outputs 0.5 everywhere — cannot distinguish real from fake
★	Training alternates: first train D (freeze G), then train G (freeze D)
★	Generator's true goal: make D(G(z)) → 1 (fool the discriminator)

Variational Autoencoder (VAE) vs Generative Adversarial Network (GAN)

Both **VAEs** and **GANs** are generative models used to create new data samples such as images, audio, or text.
However, they work very differently.

---

1. Basic Idea

Aspect	VAE	GAN
Full Form	Variational Autoencoder	Generative Adversarial Network
Main Idea	Learn probability distribution of data	Generator competes against discriminator
Architecture	Encoder + Decoder	Generator + Discriminator
Training Style	Reconstruction-based	Adversarial training

2. Working Principle

Variational Autoencoder (VAE)

A VAE has:

- **Encoder**: compresses input into latent variables
- **Latent space**: probabilistic representation
- **Decoder**: reconstructs data from latent vector

Flow:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

Goal:

- reconstruct input accurately,
- make latent space smooth and continuous.

Generative Adversarial Network (GAN)

GAN contains two networks:

Generator (G)

Creates fake samples from random noise.

Discriminator (D)

Tries to distinguish:

- real data
- fake generated data

They compete in a minimax game.

Flow:

$$z \rightarrow G(z) \rightarrow \text{fake image}$$

Discriminator checks:

$$D(x) \rightarrow \text{real/fake}$$

3. Objective Functions

VAE Loss

Two parts:

Reconstruction Loss

Measures reconstruction quality.

KL Divergence

Makes latent distribution close to normal distribution.

Overall:

$$\mathcal{L} = \text{Reconstruction Loss} + \text{KL Divergence}$$

GAN Loss

Generator tries to fool discriminator.

Discriminator tries to classify correctly.

Classic GAN objective:

$$\min_G \max_D \Big[\log D(x) + \log(1 - D(G(z))) \Big]$$

4. Output Quality

Feature	VAE	GAN
Image Sharpness	Often blurry	Very sharp and realistic
Diversity	Good	Can suffer mode collapse
Stability	Stable training	Difficult training
Sample Quality	Moderate	Excellent

5. Latent Space

Aspect	VAE	GAN
Structured latent space	Yes	Usually less structured
Easy interpolation	Yes	Harder
Representation learning	Strong	Moderate

VAEs are better when latent representation matters.

6. Training Difficulty

Aspect	VAE	GAN
Easier to train	Yes	No
Training instability	Low	High
Hyperparameter sensitivity	Lower	Higher

GANs may suffer from:

- mode collapse,
- non-convergence,
- unstable gradients.

7. Applications

VAE Applications

- Data compression
- Representation learning
- Anomaly detection
- Denoising
- Drug discovery

GAN Applications

- Photorealistic image generation
- Deepfakes
- Super-resolution
- Image-to-image translation
- Art generation

8. Advantages and Disadvantages

VAE

Advantages

- Stable training
- Meaningful latent space
- Easy interpolation

Disadvantages

- Blurry outputs
- Less realistic images

GAN

Advantages

- Highly realistic outputs
- Excellent image quality

Disadvantages

- Hard to train
- Mode collapse
- No explicit encoder in basic GAN

UNIT 5: TRAINING GANs

Key Concepts

Concept	Explanation
Minimax Game	G and D play a zero-sum game: D's gain = G's loss
Nash Equilibrium	Neither player can improve by changing strategy alone; D(x)=0.5 for all x
Convergence	Ideally G learns Pdata exactly; in practice, very hard to achieve

Training Issues

Problem	What Happens?	Fix
Mode Collapse	Generator only produces a few types of outputs (no diversity). E.g. generates only one digit.	Minibatch discrimination, unrolled GANs, WGAN
Vanishing Gradients	D becomes too strong; G gets no useful gradient signal to improve	Use -log D(G(z)) instead of log(1-D(G(z))); WGAN
Non-convergence	G and D oscillate without reaching Nash equilibrium; training unstable	Careful learning rate, gradient penalty, different training ratios

Improvement Techniques

- **Label Smoothing**: use 0.9 instead of 1 for real labels → reduces overconfident D
- **Feature Matching**: train G to match feature statistics of real data in D's intermediate layers
- **Batch Normalization**: normalize activations in each layer → stabilizes training
- **Gradient Penalty**: penalize large gradients in D (used in WGAN-GP)

📋 Exam Focus — Unit 5

★	Mode collapse definition + example: G only generates '3' when trained on MNIST digits
★	Vanishing gradient in GANs: when D is too good, log(1-D(G(z))) → 0 so G gets no gradient
★	Nash equilibrium in GAN context: G ≈ Pdata, D outputs 0.5 everywhere
★	Label smoothing: soft labels (0.9 instead of 1) prevent D from becoming overconfident
★	Which variant (WGAN, cGAN) fixes which problem — conceptual understanding

UNIT 6: ADVANCED GAN VARIANTS

▸ Conditional GAN (cGAN)

IDEA	Feed a class label y to both G and D. Generator creates class-specific images. e.g. 'Generate a handwritten 7'

$$\min_G \max_D \; V(D,G) = \mathbb{E}[\log D(x|y)] + \mathbb{E}[\log(1 - D(G(z|y)))]$$

Regular GAN	Conditional GAN (cGAN)
Input to G: noise z only	Input to G: noise z + label y
G generates random samples	G generates label-specific samples
Cannot control output class	Can say: 'Generate a cat / dog / car'
Example: random face generation	Example: text-to-image (label = text)

▸ CycleGAN — Image-to-Image Translation

IDEA	Translate between two domains WITHOUT paired training data. e.g. Horse ↔ Zebra, Summer ↔ Winter photos

Domain X (e.g. Horse photo)

▼

Generator G: X → Y (Horse → Zebra)

▼

Domain Y (e.g. Zebra photo)

▼

Generator F: Y → X (Zebra → Horse back)

▼

CYCLE CONSISTENCY: $F(G(x)) \approx x$ $\|F(G(x)) - x\|_1$ is minimised

Main Idea

CycleGAN uses:

Two Generators

$G : X \to Y$

Converts:

- Horse → Zebra

$F : Y \to X$

Converts:

- Zebra → Horse

Two Discriminators

D_Y

Checks whether image is:

- real zebra
- fake zebra

D_X

Checks whether image is:

- real horse
- fake horse

Architecture Overview

```
Horse Image
     |
     v
Generator G
(Horse → Zebra)
     |
 Fake Zebra
     |
Discriminator DY
(real or fake?)

--------------------------------

Zebra Image
     |
     v
Generator F
(Zebra → Horse)
     |
 Fake Horse
     |
Discriminator DX
(real or fake?)
```

Why “Cycle” GAN?

The key innovation is **cycle consistency**.

If:

$$\text{Horse} \xrightarrow{G} \text{Zebra}$$

then converting back should recover the original image:

$$\text{Horse} \xrightarrow{G} \text{Zebra} \xrightarrow{F} \text{Horse}$$

The reconstructed horse should look similar to the original horse.

This is called the **cycle consistency loss**.

Other GAN Variants

Variant	Key Feature	Solves
DCGAN	Deep Convolutional GAN — uses Conv/Deconv layers instead of FC layers; batch norm	Training instability for image generation
WGAN	Wasserstein GAN — uses Earth Mover Distance instead of Jensen-Shannon divergence	Mode collapse & vanishing gradients

📋 Exam Focus — Unit 6

★	cGAN vs GAN: input label y is fed to both G and D; enables conditional generation
★	CycleGAN: unpaired image translation; cycle consistency loss: F(G(x)) ≈ x
★	DCGAN: uses convolutional layers, batch normalization — key architecture differences from vanilla GAN
★	WGAN: replaces cross-entropy loss with Wasserstein distance (EMD) — fixes mode collapse
★	Know which variant solves which problem for case-based/MCQ questions

UNIT 7: APPLICATIONS OF GENERATIVE MODELS

Use Cases

Application	Model Used	Example
Image Generation	GAN, VAE, Diffusion Models	DALL-E, Stable Diffusion, Midjourney
Style Transfer	CycleGAN, Neural Style Transfer	Make photo look like Monet painting
Super-Resolution	SRGAN (Super Resolution GAN)	Upscale blurry 64×64 image to HD 256×256
Data Augmentation	GAN, VAE	Generate extra training samples for rare diseases
Text-to-Image	cGAN, Diffusion Models	DALL-E 3: 'a cat in space wearing a hat'
Medical Imaging	GAN, VAE, Diffusion Models	Synthetic MRI/X-ray scans for training; anomaly detection

Real-World Examples

Deepfakes	GANs swap faces in video. FaceSwap, DeepFaceLab. Major ethical concerns — fake news, fraud.
AI Art	Midjourney, DALL-E, Stable Diffusion generate images from text prompts. StyleGAN creates realistic faces.
Medical	Generate synthetic patient data for rare diseases. Detect tumours using AE anomaly detection. Denoise MRI scans.

📋 Exam Focus — Unit 7

★	Model-to-application mapping: Know which model → which task (e.g. CycleGAN → style transfer/unpaired translation)
★	Super-resolution: SRGAN takes low-res input → generates high-res output using GAN
★	Data augmentation: why it helps (small dataset) and how (generate synthetic samples)
★	Deepfakes: which technique used and what ethical concerns arise
★	Case questions: given a task, identify whether AE / VAE / GAN / cGAN is most appropriate

QUICK REVISION SUMMARY

Model	Core Idea	Key Formula / Concept
AE	Compress & reconstruct data through bottleneck	`Loss = MSE(x, x^)`
VAE	Encode to distribution; reparameterize; generate by sampling	`Loss = Recon + KL | z=μ+σε`
GAN	Generator vs Discriminator adversarial game	`min_G max_D [logD(x)+log(1-D(G(z)))]`
cGAN	Conditional generation by feeding label y to G and D	`G(z|y) and D(x|y)`
CycleGAN	Unpaired image-to-image translation using cycle consistency	`||F(G(x)) - x||₁ ≈ 0`

Good luck on your exam! 🎯